Nonconvex Zeroth-Order Stochastic ADMM Methods with Lower Function Query Complexity

Feihu Huang, Shangqian Gao, Jian Pei, *Fellow, IEEE*, and Heng Huang

Abstract—Zeroth-order (a.k.a, derivative-free) methods are a class of effective optimization methods for solving complex machine learning problems, where gradients of the objective functions are not available or computationally prohibitive. Recently, although many zeroth-order methods have been developed, these approaches still have two main drawbacks: 1) high function query complexity; 2) not being well suitable for solving the problems with complex penalties and constraints. To address these challenging drawbacks, in this paper, we propose a class of faster zeroth-order stochastic alternating direction method of multipliers (ADMM) methods (ZO-SPIDER-ADMM) to solve the nonconvex finite-sum problems with multiple nonsmooth penalties. Moreover, we prove that the ZO-SPIDER-ADMM methods can achieve a lower function query complexity of $O(nd + dn^{\frac{1}{2}}\epsilon^{-1})$ for finding an ϵ-stationary point, which improves the existing best nonconvex zeroth-order ADMM methods by a factor of $O(d^{\frac{1}{3}}n^{\frac{1}{6}})$, where n and d denote the sample size and data dimension, respectively. At the same time, we propose a class of faster zeroth-order online ADMM methods (ZOO-ADMM+) to solve the nonconvex online problems with multiple nonsmooth penalties. We also prove that the proposed ZOO-ADMM+ methods achieve a lower function query complexity of $O(d\epsilon^{-\frac{3}{2}})$, which improves the existing best result by a factor of $O(\epsilon^{-\frac{1}{2}})$. Extensive experimental results on the structure adversarial attack on black-box deep neural networks demonstrate the efficiency of our new algorithms.

Index Terms—Zeroth-Order, Derivative-Free, ADMM, Nonconvex, Black-Box Adversarial Attack.

1 Introduction

Zeroth-order (*a.k.a*, derivative-free, gradient-free) methods [1], [2] are a class of powerful optimization tools for many machine learning problems, which only need function values (not gradient) in the optimization. For example, zeroth-order optimization methods have been applied to bandit feedback analysis [3], reinforcement learning [4], and adversarial attacks on black-box deep neural networks (DNNs) [5], [6]. Thus, recently the zeroth-order methods have been increasingly studied. For example, [7] proposed the zeroth-order gradient descent methods based on the Gaussian smoothing gradient estimator. [8] presented the zeroth-order stochastic gradient (ZO-SGD) methods for stochastic optimization. [9] proposed a class of zeroth-order proximal stochastic gradient methods for stochastic optimization with nonsmooth penalties. Subsequently, [10], [11] proposed the zeroth-order online (or stochastic) alternating direction method of multipliers (ADMM) methods to also solve stochastic optimization with nonsmooth penalties. At the same time, [12] developed a class of zeroth-order conditional gradient methods for constrained optimization.

- *Feihu Huang is with the Department of Electrical and Computer Engineering, University of Pittsburgh, USA, and is also with College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, MIIT Key Laboratory of Pattern Analysis and Machine Intelligence, Nanjing, China. E-mail: huangfeihu2018@gmail.com*
- *Shangqian Gao is with the Department of Electrical and Computer Engineering, University of Pittsburgh, Pittsburgh, USA. E-mail: shg84@pitt.edu*
- *Jian Pei is with the Department of Computer Science, Duke University, Durham, USA. E-mail: j.pei@duke.edu*
- *Heng Huang is with the Department of Computer Science, University of Maryland College Park, USA. E-mail: heng@umd.edu*

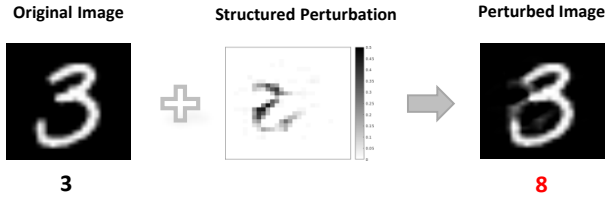


Fig. 1. Structured perturbation example not only fools the DNN, but also has some explicable. Black and red labels denote the initial label and the label after attack, respectively.

These zeroth-order methods frequently suffer from slow convergence rate and high function query complexity due to high variances generated from estimating zeroth-order (stochastic) gradients. To alleviate this issue, some accelerated zeroth-order methods have been developed. For example, to accelerate the ZO-SGD, [6], [14] proposed a class of faster zeroth-order stochastic variance-reduced gradient (ZO-SVRG) methods by using the SVRG [15], [16], [17]. To reduce function query complexity, the SPIDER-SZO [18] and ZO-SPIDER-Coord [19] have been proposed by using stochastic path-integrated differential estimator, *i.e.*, SARAH/SPIDER [18], [20], [21]. For big data optimization, asynchronous parallel zeroth-order methods [22], [23] and distributed zeroth-order methods [24] have been developed.

To simultaneously deal with the non-convex loss and non-smooth regularization, [9], [25] proposed some zeroth-

TABLE 1
Convergence property comparison of the zeroth-order ADMM algorithms for finding an ϵ-stationary point. C, NC, S, NS and mNS are the abbreviations of convex, non-convex, smooth, non-smooth and the sum of multiple non-smooth functions, respectively. d is the dimension of data and n denotes the sample size. GauGE, UniGE and CooGE are abbreviations of Gaussian distribution, Uniform distribution and Coordinate-wise smoothing gradient estimators, respectively.

Type	Algorithm	Gradient Estimator	Reference	Problem	Function Query Complexity
Finite-sum	ZO-SVRG-ADMM	CooGE	[13]	NC(S) + C(mNS)	$O(nd + d^2 n^{\frac{2}{3}} \epsilon^{-1})$
	ZO-SAGA-ADMM	CooGE			$O(nd + d^{\frac{4}{3}} n^{\frac{2}{3}} \epsilon^{-1})$
	ZO-SPIDER-ADMM	CooGE	Ours	NC(S) + C(mNS)	$O(nd + dn^{\frac{1}{2}} \epsilon^{-1})$
	ZO-SPIDER-ADMM	CooGE+UniGE			
Online	ZOO-ADMM	GauGE	[10]	C(S) + C(NS)	$O(d\epsilon^{-2})$
	ZO-GADM	UniGE	[11]		
	ZOO-ADMM+	CooGE	Ours	NC(S) + C(mNS)	$O(d\epsilon^{-\frac{3}{2}})$
	ZOO-ADMM+	CooGE+UniGE			

order proximal stochastic gradient methods. However, these nonconvex zeroth-order methods still are not competent for many machine learning problems with complex nonsmooth penalties and constraints, such as the structured adversarial attack to (black-box) DNNs in [13], [26] (see Fig. 1). More recently, thus, [13] proposed a class of nonconvex zeroth-order stochastic ADMM methods (*i.e.*, ZO-SVRG-ADMM and ZO-SAGA-ADMM) to solve these complex problems. However, these zeroth-order ADMM methods suffer from high function query complexity (please see in Table 1).

In this paper, thus, we propose a class of faster zeroth-order stochastic ADMM methods with lower function query complexity to solve the following nonconvex nonsmooth problem:

$$\begin{aligned} \min_{x\in\mathbb{R}^d, \{y_j\in\mathbb{R}^p\}_{j=1}^m} \quad & f(x) + \sum_{j=1}^m \psi_j(y_j) \\ \text{s.t.} \quad & Ax + \sum_{j=1}^m B_j y_j = c, \end{aligned} \tag{1}$$

$$f(x) := \begin{cases} \dfrac{1}{n}\sum_{i=1}^n f_i(x) & \text{(finite-sum)} \\ \mathbb{E}_\xi[f(x;\xi)] & \text{(online)} \end{cases}$$

where $A \in \mathbb{R}^{l\times d}$, $B_j \in \mathbb{R}^{l\times p}$ for $j \in [m]$, $c \in \mathbb{R}^l$, and ξ denotes a random variable that following an unknown distribution. The equality constraint in the problem (1) encodes structure pattern of model parameters. Here $f(x) : \mathbb{R}^d \to \mathbb{R}$ is a *nonconvex* and smooth function, and $\psi_j(y_j) : \mathbb{R}^p \to \mathbb{R}$ is a convex and possibly *nonsmooth* function for all $j \in [m]$, $m \geq 1$. Here the explicit gradients of $f_i(x)$ and $f(x;\xi)$ are difficult or infeasible to obtain. Under this case, we need to use the zeroth-order gradient estimators [6], [7] to estimate gradients of $f_i(x)$ and $f(x;\xi)$. For the problem (1), its *finite-sum* subproblem generally comes from the empirical loss minimization in machine learning, and its *online* subproblem generates from the expected loss minimization. Here the non-smooth regularization functions $\{\psi_j(y_j)\}_{j=1}^m$ can encode some complex superposition structures such as sparse and low-rank structure in robust principal component analysis (PCA) [27], [28], and group-wise sparsity and within group sparsity structure in sparse group lasso [29].

In fact, the problem (1) includes many machine learning problems. For example, when $m = 1$, $A = I_d$, $B_1 = -I_d$ and $c = 0$, the problem (1) will reduce to the standard *regularized* risk minimization problem such as training deep neural networks (DNNs), defined as

$$\min_{x\in\mathbb{R}^d} f(x) + \psi_1(x), \tag{2}$$

where $f(x)$ is a nonconvex loss function, and $\psi_1(x)$ is a regularization such as $\psi_1(x) = \tau\|x\|^2$ with $\tau > 0$. When $m = 1$, $A \neq I_d$ denotes a sparse graph correlation matrix, $B_1 = -I_d$ and $c = 0$, i.e., the equality constraint $Ax = y_1$, and $\psi_1(y_1) = \tau\|y_1\|_1$, the problem (1) will degenerate to the risk minimization problem with graph-guided fused lasso [30] regularization, defined as

$$\min_{x\in\mathbb{R}^d} f(x) + \tau\|Ax\|_1, \tag{3}$$

where $f(x)$ is a loss function, and $\tau > 0$. Because the term $\|Ax\|_1$ consider the sparse relationships between different features, it can improve the performances of tasks such as SVM [31]. When $m = 2$, $A = I_{d_1}$, $B_1 = -I_{d_1}$, $B_2 = -I_{d_1}$ and $c = 0$, the problem (1) will reduce to a risk minimization problem with *two* penalties such as sparse and low-rank structure in robust PCA [27], [28], defined as

$$\begin{aligned} \min_{X,Y_1,Y_2} \quad & f(X) + \tau_1\|Y_1\|_1 + \tau_2\|Y_2\|_*, \\ \text{s.t.} \quad & X = Y_1 + Y_2, \end{aligned} \tag{4}$$

where $X, Y_1, Y_2 \in \mathbb{R}^{d_1\times d_2}$, and $\tau_1, \tau_2 > 0$ denote the tuning parameters. Here we can use some nonconvex functions $f(X)$ such as $\|X - D\|_\gamma$ $(0 < \gamma < 1)$ instead of $\|X - D\|_2^2$ used in [27], [28], where $D \in \mathbb{R}^{d_1\times d_2}$ is data matrix.

It is worth highlighting that our **main contributions** are four-fold:

1) We propose a class of new faster zeroth-order stochastic ADMM (*i.e.*, ZO-SPIDER-ADMM) methods to solve the finite-sum problem (1), based on variance reduced technique of SPIDER/SARAH and different zeroth-order gradient estimators.
2) We prove that the ZO-SPIDER-ADMM methods reach a lower function query complexity of $O(dn + dn^{\frac{1}{2}}\epsilon^{-1})$ for finding an ϵ-stationary point, which improves the existing best nonconvex zeroth-order ADMM methods by a factor of $O(d^{\frac{1}{3}} n^{\frac{1}{6}})$.

3) We extend the ZO-SPIDER-ADMM methods to the online setting, and propose a class of faster zeroth-order online ADMM methods (*i.e.*, ZOO-ADMM+) to solve the online problem (1).
4) We provide the theoretical analysis on the convergence properties of our ZOO-ADMM+ methods, and prove that they achieve a lower function query complexity of $O(d\epsilon^{-\frac{3}{2}})$, which improves the existing best result by a factor of $O(\epsilon^{-\frac{1}{2}})$.

Notations. To make the paper easier to follow, we give the following notations:

- $[m] = \{1, 2, \cdots, m\}$ and $[j : m] = \{j, j+1, \cdots, m\}$ for all $1 \leq j \leq m$.
- $\|\cdot\|$ denotes the vector ℓ_2 norm and the matrix spectral norm, respectively.
- $\|x\|_G = \sqrt{x^T G x}$, where G is a positive definite matrix.
- $\sigma^A_{\min}$ and $\sigma^A_{\max}$ denote the minimum and maximum eigenvalues of $A^T A$, respectively.
- $\sigma^{B_j}_{\max}$ denotes the maximum eigenvalues of $B_j^T B_j$ for all $j \in [m]$, and $\sigma^B_{\max} = \max_{1\leq j\leq m} \sigma^{B_j}_{\max}$.
- $\sigma_{\min}(G)$ and $\sigma_{\max}(G)$ denote the minimum and maximum eigenvalues of matrix G, respectively; the conditional number $\kappa_G = \frac{\sigma_{\max}(G)}{\sigma_{\min}(G)}$.
- $\sigma_{\min}(H_j)$ and $\sigma_{\max}(H_j)$ denote the minimum and maximum eigenvalues of matrix H_j for all $j \in [m]$, respectively; $\sigma_{\min}(H) = \min_{1\leq j\leq m} \sigma_{\min}(H_j)$ and $\sigma_{\max}(H) = \max_{1\leq j\leq m} \sigma_{\max}(H_j)$.
- $\mu,\ \nu$ denote the smoothing parameters of zeroth-order gradient estimators.
- η denotes the step size of updating variable x.
- L denotes the Lipschitz constant of $\nabla f(x)$.
- $b,\ b_1,\ b_2$ denote the mini-batch sizes of stochastic zeroth-order gradients.

2 Related Work

In the section, we overview some representative existing ADMM methods and zeroth-order methods, respectively.

2.1 ADMM Methods

ADMM [32], [33] is a popular optimization method for solving the composite and constrained problems. For example, due to the flexibility in splitting the objective function into loss and complex penalty, the ADMM can relatively easily solve some problems with complicated structure penalty such as the graph-guided fused lasso [30], which are too complicated for the other popular optimization methods such as proximal gradient methods [34]. Thus, recently many ADMM methods [35], [36], [37], [38], [39] have been proposed and studied. For example, the classic convergence analysis of ADMM has been studied in [35]. Subsequently, some stochastic (or online) ADMM methods [31], [40], [41], [42], [43], [44] have been proposed by visiting only one or mini-batch samples instead of all samples. In fact, the ADMM method is also successful in solving many nonconvex machine learning problems such as training neural networks [45]. Thus, the nonconvex ADMM methods [46], [47], [48], [49] have been widely studied. Subsequently, the nonconvex stochastic ADMM methods [50], [51] have been proposed. In particular, [51] first studied the gradient (or sample) complexities of the nonconvex stochastic ADMM methods.

2.2 Zeroth-Order Methods

Zeroth-order methods have been increasingly attracted due to effectively solve some complex machine learning problems, whose the explicit gradients are difficult or even infeasible to access. For example, [7], [8], [52] proposed several zeroth-order algorithms (*i.e.*, ZO-SGD and ZO-SCD) based on the Gaussian smoothing technique. Subsequently, some accelerated zeroth-order stochastic methods (*i.e.*, ZO-SVRG and ZO-SPIDER) [6], [18], [19] have been proposed by using the variance reduced techniques of SVRG and SPIDER, respectively. To solve the nonsmooth optimization, several zeroth-order proximal algorithms [9], [25] have been proposed. At the same time, the zeroth-order ADMM-type algorithms [10], [11], [13] also have been proposed.

3 Faster Zeroth-Order ADMMs

In this section, we propose a class of faster zeroth-order stochastic ADMM methods (*i.e.*, ZO-SPIDER-ADMM with different variants) to solve the non-convex finite-sum problem (1). At the same time, we extend the ZO-SPIDER-ADMM methods to the online setting and propose a faster zeroth-order online ADMM methods (*i.e.*, ZOO-ADMM+) to solve the online problem (1).

3.1 Preliminary

We first introduce an augmented Lagrangian function of the problem (1) as follows:

$$\mathcal{L}_\rho(x, y_{[m]}, \lambda) = \Big\{ f(x) + \sum_{j=1}^m \psi_j(y_j) - \langle \lambda, Ax + \sum_{j=1}^m B_j y_j - c\rangle + \frac{\rho}{2}\|Ax + \sum_{j=1}^m B_j y_j - c\|^2 \Big\}, \tag{5}$$

where $f(x) = \frac{1}{n}\sum_{i=1}^n f_i(x)$ or $f(x) = \mathbb{E}[f(x;\xi)]$, and $\lambda \in \mathbb{R}^l$ denotes the dual variable, and $\rho > 0$ denotes the penalty parameter.

In the problem (1), the explicit expression of gradients for $f_i(x)$ and $f(x;\xi_i)$ for all i are not available, and only its function values are available. We can use the coordinate smoothing gradient estimator (CooGE) [6], [19] to evaluate gradient:

$$\hat{\nabla}_{\text{coo}} f_i(x) = \sum_{j=1}^d \frac{f_i(x+\mu e_j) - f_i(x-\mu e_j)}{2\mu} e_j, \tag{6}$$

$$\hat{\nabla}_{\text{coo}} f(x;\xi_i) = \sum_{j=1}^d \frac{f(x+\mu e_j;\xi_i) - f(x-\mu e_j;\xi_i)}{2\mu} e_j \tag{7}$$

where $\mu > 0$ is a coordinate-wise smoothing parameter, and e_j is a standard basis vector with 1 at its j-th coordinate, and 0 otherwise. Meanwhile, we can also apply the uniform

smoothing gradient estimator (UniGE) [6], [19] to evaluate gradient:

$$\hat{\nabla}_{\texttt{uni}} f_i(x) = \frac{d(f_i(x+\nu u) - f_i(x))}{\nu} u, \tag{8}$$

$$\hat{\nabla}_{\texttt{uni}} f(x;\xi_i) = \frac{d(f(x+\nu u;\xi_i) - f(x;\xi_i))}{\nu} u, \tag{9}$$

where $\nu > 0$ is a smoothing parameter and $u \in \mathbb{R}^d$ is a vector generated from the uniform distribution over the unit sphere. In addition, we define $f_\nu(x) = \mathbb{E}_{u\sim U_B}[f(x+\nu u)]$ be a smooth approximation of $f(x)$, where U_B is the uniform distribution over d-dimensional unit Euclidean sphere B.

3.2 ZO-SPIDER-ADMM Algorithms

In this subsection, we propose a class of faster zeroth-order SPIDER-ADMM methods (*i.e.*, ZO-SPIDER-ADMM) to solve the finite-sum problem (1). The naive extension of the multi-block ADMM method may diverge [53], however, our method use the linearized technique as in [49] to guarantee its convergence. We first propose the ZO-SPIDER-ADMM (CooGE) algorithm based on the CooGE gradient estimator and variance reduced technique of SPIDER/SARAH, which is described in Algorithm 1.

Algorithm 1 ZO-SPIDER-ADMM (CooGE) Algorithm

1: **Input:** Total iteration K, mini-batch size b, epoch size q, penalty parameter ρ and step size η;
2: **Initialize:** $x_0 \in \mathbb{R}^d$, $y_j^0 \in \mathbb{R}^p$, $j \in [m]$ and $\lambda_0 \in \mathbb{R}^l$;
3: **for** $k = 0, 1, \cdots, K-1$ **do**
4: **if** $\mod(k,q) = 0$ **then**
5: Compute $v_k = \frac{1}{n}\sum_{i=1}^n \hat{\nabla}_{\texttt{coo}} f_i(x_k)$;
6: **else**
7: Uniformly randomly pick a mini-batch $\mathcal{S}$ ($|\mathcal{S}| = b$) from $\{1, 2, \cdots, n\}$ with replacement, then update $v_k = \frac{1}{b}\sum_{i\in\mathcal{S}}\big(\hat{\nabla}_{\texttt{coo}} f_i(x_k) - \hat{\nabla}_{\texttt{coo}} f_i(x_{k-1})\big) + v_{k-1}$;
8: **end if**
9: $y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k)$ defined in (10), for all $j \in [m]$;
10: $x_{k+1} = \arg\min_{x\in\mathbb{R}^d} \hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k)$ defined in (12);
11: $\lambda_{k+1} = \lambda_k - \rho(Ax_{k+1} + \sum_{j=1}^m B_j y_j^{k+1} - c)$;
12: **end for**
13: **Output (in theory):** $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ chosen uniformly randomly from $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$.
14: **Output (in practice):** $\{x_K, y_{[m]}^K, z_K\}$.

At the step 9 of Algorithm 1, we update the variables $\{y_j\}_{j=1}^m$ by solving the following sub-problem, for all $j \in [m]$

$$y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k), \tag{10}$$

$$\tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k) = \Big\{ f(x_k) + \sum_{i=1}^{j-1}\psi_i(y_i^{k+1}) + \psi_j(y_j) + \sum_{i=j+1}^{m}\psi_j(y_j^k) - \lambda_k^T(B_j y_j + \tilde{c}_j) + \frac{\rho}{2}\|B_j y_j + \tilde{c}\|^2 + \frac{1}{2}\|y_j - y_j^k\|_{H_j}^2 \Big\},$$

where $\tilde{c}_j = Ax_k + \sum_{i=1}^{j-1} B_i y_i^{k+1} + \sum_{i=j+1}^m B_i y_i^k - c$ and $H_j \succ 0$. Here $\tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k)$ is an approximated function of $\mathcal{L}_\rho(x, y_{[m]}, \lambda)$ over x_k, $y_{[j-1]}^{k+1}$ and $y_{[j:m]}^k$, where the term $\frac{1}{2}\|y_j - y_j^k\|_{H_j}^2$ ensures that the solution y_j^{k+1} is not far from y_j^k. When set $H_j = r_j I_p - \rho B_j^T B_j \succeq I_p$ with $r_j \geq \rho\sigma_{\max}(B_j^T B_j) + 1$ for all $j \in [m]$ to linearize the term $\frac{\rho}{2}\|B_j y_j + \tilde{c}\|^2$, then we can use the following proximal operator to update y_j, for all $j \in [m]$

$$y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \frac{1}{2}\|y_j - w_j^k\|^2 + \frac{1}{r_j}\psi_j(y_j), \tag{11}$$

where $w_j^k = \frac{1}{r_j}\big(H_j y_j^k - \rho B_j^T\tilde{c} + B_j^T\lambda_k\big)$. For example, when $\psi_j(y_j) = \|y_j\|_1$, this subproblem (11) has a closed-form solution $y_j^{k+1} = \text{sign}(w_j^k)\max\big(|w_j^k| - \frac{1}{r_j}, 0\big)$.

At the step 10 of Algorithm 1, we update the variable x by solving the following problem:

$$x_{k+1} = \arg\min_{x\in\mathbb{R}^d} \hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k) \tag{12}$$

$$\hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k) = \Big\{ f(x_k) + v_k^T(x - x_k) + \frac{1}{2\eta}\|x - x_k\|_G^2 + \sum_{j=1}^m \psi_j(y_j^{k+1}) - \lambda_k^T(Ax + \sum_{j=1}^m B_j y_j^{k+1} - c) + \frac{\rho}{2}\|Ax + \sum_{j=1}^m B_j y_j^{k+1} - c\|^2 \Big\},$$

where $\eta > 0$ is a step size and $G \succ 0$. Here $\hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k)$ is an approximated function of $\mathcal{L}_\rho(x, y_{[m]}, \lambda)$ over x_k and $\{y_j^{k+1}\}_{j=1}^m$, where the term $\frac{1}{2\eta}\|x - x_k\|_G^2$ ensures that the solution x_{k+1} is not far from x_k. By solving the problem (12), we can obtain:

$$x_{k+1} = \tilde{A}^{-1}\Big(\frac{G}{\eta}x_k - v_k - \rho A^T(\sum_{j=1}^m B_j y_j^{k+1} - c - \frac{\lambda_k}{\rho})\Big),$$

where $\tilde{A} = \frac{G}{\eta} + \rho A^T A$. To avoid computing inverse of matrix $\tilde{A}$, we can set $G = rI_d - \rho\eta A^T A \succeq I_d$ with $r \geq \rho\eta\sigma_{\max}^A + 1$ to linearize term $\frac{\rho}{2}\|Ax + \sum_{j=1}^m B_j y_j^{k+1} - c\|^2$. Then we have

$$x_{k+1} = \frac{Gx_k}{r} - \frac{\eta v_k}{r} - \frac{\eta\rho}{r}A^T(\sum_{j=1}^m B_j y_j^{k+1} - c - \frac{\lambda_k}{\rho}). \tag{13}$$

In Algorithm 1, we use the following semi-stochastic zeroth-order gradient to update x:

$$v_k = \begin{cases} \frac{1}{n}\sum_{i=1}^n \hat{\nabla}_{\texttt{coo}} f_i(x_k), & \text{if} \quad \mod(k,q) = 0 \\ \frac{1}{b}\sum_{i\in\mathcal{S}}\big(\hat{\nabla}_{\texttt{coo}} f_i(x_k) - \hat{\nabla}_{\texttt{coo}} f_i(x_{k-1})\big) + v_{k-1}, & \text{otw.} \end{cases}$$

where **otw.** denotes otherwise. In fact, we have $\mathbb{E}_{\mathcal{S}}[v_k] = \hat{\nabla}_{\texttt{coo}} f(x_k) \neq \nabla f(x_k)$, *i.e.*, this stochastic gradient is a **biased** estimate of the true full gradient. Thus, we choose the appropriate step size η, penalty parameter ρ and smoothing parameter μ to guarantee the convergence of our algorithms, which will be discussed in the following convergence analysis.

From the above (6), the CooGE needs $2d$ function values to estimate one zeroth-order gradient. Clearly, due to relying

Algorithm 2 ZO-SPIDER-ADMM (CooGE+UniGE) Algorithm

Input: Total iteration K, mini-batch size b, epoch size q, penalty parameter ρ and step size η;
Initialize: $x_0 \in \mathbb{R}^d$, $y_j^0 \in \mathbb{R}^p,\ j \in [m]$ and $\lambda_0 \in \mathbb{R}^l$;
for $k = 0, 1, \cdots, K-1$ **do**
  if $\mod(k, q) = 0$ **then**
    Compute $v_k = \frac{1}{n}\sum_{i=1}^n \hat{\nabla}_{\texttt{coo}} f_i(x_k)$;
  else
    Uniformly randomly pick a mini-batch $\mathcal{S}$ ($|\mathcal{S}| = b$) from $\{1, 2, \cdots, n\}$ with replacement, and draw i.i.d. $\{u_1, \cdots, u_b\}$ from uniform distribution over unit sphere, then update $v_k = \frac{1}{b}\sum_{i\in\mathcal{S}} \big(\hat{\nabla}_{\texttt{uni}} f_i(x_k) - \hat{\nabla}_{\texttt{uni}} f_i(x_{k-1})\big) + v_{k-1}$;
  end if
  $y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k)$ defined in (10), for all $j \in [m]$;
  $x_{k+1} = \arg\min_{x\in\mathbb{R}^d} \hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k)$ defined in (12);
  $\lambda_{k+1} = \lambda_k - \rho(Ax_{k+1} + \sum_{j=1}^m B_j y_j^{k+1} - c)$;
end for
Output (in theory): $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ chosen uniformly randomly from $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$.
Output (in practice): $\{x_K, y_{[m]}^K, z_K\}$.

on the CooGE, Algorithm 1 has an expensive computation cost. To alleviate this issue, we use the UniGE to replace part of the CooGE use in Algorithm 1, since the UniGE only requires two function values to estimate one zeroth-order gradient in the above (8). Thus, we propose a ZO-SPIDER-ADMM (CooGE+UniGE) algorithm based on the mixture of CooGE and UniGE gradient estimators. Algorithm 2 shows the ZO-SPIDER-ADMM (CooGE+UniGE) algorithm. In Algorithm 2, we use the following zeroth-order gradient estimator

$$v_k = \begin{cases} \dfrac{1}{n}\displaystyle\sum_{i=1}^n \hat{\nabla}_{\texttt{coo}} f_i(x_k), & \text{if} \quad \mod(k,q) = 0 \\ \dfrac{1}{b}\displaystyle\sum_{i\in\mathcal{S}} \big(\hat{\nabla}_{\texttt{uni}} f_i(x_k) - \hat{\nabla}_{\texttt{uni}} f_i(x_{k-1})\big) + v_{k-1}, & \text{otw.} \end{cases}$$

Then updating the parameters $\{x, y_{[m]}, \lambda\}$ is the same as the above algorithm 1.

3.3 ZOO-ADMM+ Algorithms

In this subsection, we extend the above ZO-SPIDER-ADMM methods to the online setting and propose a class of faster zeroth-order online ADMM (*i.e.*, ZOO-ADMM+) method to solve the online problem (1). We begin with proposing the ZOO-ADMM+(CooGE) algorithm based on the CooGE gradient estimator, which is described in Algorithm 3.

Under the online setting, $f(x) = \mathbb{E}_\xi[f(x;\xi)]$ denotes a population risk over an underlying data distribution. The online problem (1) can be viewed as having infinite samples, so we are not able to estimate zeroth-order gradient of the function $f(x)$. Thus, we estimate the mini-batch zeroth-order gradient instead of the full zeroth-order gradient. In

Algorithm 3 ZOO-ADMM+(CooGE) Algorithm

Input: Total iteration K, mini-batch sizes $b_1,\ b_2$, epoch size q, penalty parameter ρ and step size η;
Initialize: $x_0 \in \mathbb{R}^d$, $y_j^0 \in \mathbb{R}^p,\ j \in [m]$ and $\lambda_0 \in \mathbb{R}^l$;
for $k = 0, 1, \cdots, K-1$ **do**
  if $\mod(k, q) = 0$ **then**
    Draw independently $\mathcal{S}_1$ ($|\mathcal{S}_1| = b_1$) samples $\{\xi_i\}_{i=1}^{b_1}$, and compute $v_k = \frac{1}{b_1}\sum_{i\in\mathcal{S}_1} \hat{\nabla}_{\texttt{coo}} f(x_k;\xi_i)$;
  else
    Draw independently $\mathcal{S}_2$ ($|\mathcal{S}_2| = b_2$) samples $\{\xi_i\}_{i=1}^{b_2}$, and compute $v_k = \frac{1}{b_2}\sum_{i\in\mathcal{S}_2} \big(\hat{\nabla}_{\texttt{coo}} f(x_k;\xi_i) - \hat{\nabla}_{\texttt{coo}} f_i(x_{k-1};\xi_i)\big) + v_{k-1}$;
  end if
  $y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k)$ defined in (10), for all $j \in [m]$;
  $x_{k+1} = \arg\min_{x\in\mathbb{R}^d} \hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k)$ defined in (12);
  $\lambda_{k+1} = \lambda_k - \rho(Ax_{k+1} + \sum_{j=1}^m B_j y_j^{k+1} - c)$;
end for
Output (in theory): $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ chosen uniformly randomly from $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$.
Output (in practice): $\{x_K, y_{[m]}^K, z_K\}$.

Algorithm 3, we use the zeroth-order stochastic gradient as follows:

$$v_k = \begin{cases} \dfrac{1}{b_1}\displaystyle\sum_{i\in\mathcal{S}_1} \hat{\nabla}_{\texttt{coo}} f(x_k;\xi_i), & \text{if} \quad \mod(k,q) = 0 \\ \dfrac{1}{b_2}\displaystyle\sum_{i\in\mathcal{S}_2} \big(\hat{\nabla}_{\texttt{coo}} f(x_k;\xi_i) - \hat{\nabla}_{\texttt{coo}} f(x_{k-1};\xi_i)\big) + v_{k-1}, & \text{otw.} \end{cases}$$

Algorithm 4 ZOO-ADMM+(CooGE+UniGE) Algorithm

Input: Total iteration K, mini-batch sizes $b_1,\ b_2$, epoch size q, penalty parameter ρ and step size η;
Initialize: $x_0 \in \mathbb{R}^d$, $y_j^0 \in \mathbb{R}^p,\ j \in [m]$ and $\lambda_0 \in \mathbb{R}^l$;
for $k = 0, 1, \cdots, K-1$ **do**
  if $\mod(k, q) = 0$ **then**
    Draw independently $\mathcal{S}_1$ ($|\mathcal{S}_1| = b_1$) samples $\{\xi_i\}_{i=1}^{b_1}$, and compute $v_k = \frac{1}{b_1}\sum_{i\in\mathcal{S}_1} \hat{\nabla}_{\texttt{coo}} f(x_k;\xi_i)$;
  else
    Draw independently $\mathcal{S}_2$ ($|\mathcal{S}_2| = b_2$) samples $\{\xi_i\}_{i=1}^{b_2}$, and draw *i.i.d.* $\{u_1, \cdots, u_{b_2}\}$ from uniform distribution over unit sphere, then compute $v_k = \frac{1}{b_2}\sum_{i\in\mathcal{S}_2} \big(\hat{\nabla}_{\texttt{uni}} f(x_k;\xi_i) - \hat{\nabla}_{\texttt{uni}} f(x_{k-1};\xi_i)\big) + v_{k-1}$;
  end if
  $y_j^{k+1} = \arg\min_{y_j\in\mathbb{R}^p} \tilde{\mathcal{L}}_{\rho,j}(x_k, y_{[j-1]}^{k+1}, y_j, y_{[j+1:m]}^k, \lambda_k)$ defined in (10), for all $j \in [m]$;
  $x_{k+1} = \arg\min_{x\in\mathbb{R}^d} \hat{\mathcal{L}}_\rho(x, y_{[m]}^{k+1}, \lambda_k, v_k)$ defined in (12);
  $\lambda_{k+1} = \lambda_k - \rho(Ax_{k+1} + \sum_{j=1}^m B_j y_j^{k+1} - c)$;
end for
Output (in theory): $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ chosen uniformly randomly from $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$.
Output (in practice): $\{x_K, y_{[m]}^K, z_K\}$.

Similarly, we propose a ZOO-ADMM+ (CooGE+UniGE) method based on the mixture of CooGE and UniGE gra-

dient estimators. Algorithm 4 shows the ZOO-ADMM+ (CooGE+UniGE) method. In Algorithm 4, we use zeroth-order gradient estimator as follows:

$$v_k=\begin{cases}\dfrac{1}{b_1}\sum\limits_{i\in\mathcal{S}_1}\hat{\nabla}_{\texttt{coo}}f(x_k;\xi_i), & \text{if} \quad \mod(k,q)=0\\ \dfrac{1}{b_2}\sum\limits_{i\in\mathcal{S}_2}\big(\hat{\nabla}_{\texttt{uni}}f(x_k;\xi_i)-\hat{\nabla}_{\texttt{uni}}f(x_{k-1};\xi_i)\big)+v_{k-1}, & \text{otw.}\end{cases}$$

4 Theoretical Analysis

In the section, we study the convergence properties of the above proposed algorithms. All related proofs are provided in the supplementary document. We first give some standard assumptions regarding the problem (1), and define the standard ϵ-stationary point of the problem (1), as used in [49], [51].

Throughout the paper, let $c_k=\lfloor k/q\rfloor$ such that $c_kq\leq k\leq(c_k+1)q-1$. Let the sequence $\{x_k,y^k_{[m]},\lambda_k\}_{k=1}^K$ be generated from our algorithms, we define a useful variable

$$\theta_k=\mathbb{E}\big[\|x_{k+1}-x_k\|^2+\|x_k-x_{k-1}\|^2+\frac{1}{q}\sum_{i=c_kq}^{k}\|x_{i+1}-x_i\|^2+\sum_{j=1}^{m}\|y_j^k-y_j^{k+1}\|^2\big]. \tag{14}$$

Definition 1. *Given $\epsilon>0$, the point $(x^*,y^*_{[m]},\lambda^*)$ is said to be an ϵ-stationary point of the problem* (1)*, if it holds that*

$$\mathbb{E}\big[dist(0,\partial\mathcal{L}(x^*,y^*_{[m]},\lambda^*))^2\big]\leq\epsilon, \tag{15}$$

where $f(x)=\frac{1}{n}\sum_{i=1}^n f_i(x)$ or $f(x)=\mathbb{E}[f(x;\xi)]$, and $\mathcal{L}(x,y_{[m]},\lambda)=f(x)+\sum_{j=1}^m\psi_j(y_j)-\langle\lambda,Ax+\sum_{j=1}^m B_jy_j-c\rangle$, and

$$\partial\mathcal{L}(x,y_{[m]},\lambda)=\begin{bmatrix}\nabla_x\mathcal{L}(x,y_{[m]},\lambda)\\ \partial_{y_1}\mathcal{L}(x,y_{[m]},\lambda)\\ \cdots\\ \partial_{y_m}\mathcal{L}(x,y_{[m]},\lambda)\\ -Ax-\sum_{j=1}^m B_jy_j+c\end{bmatrix},$$

and $dist(0,\partial\mathcal{L})=\inf_{\mathcal{L}'\in\partial\mathcal{L}}\|0-\mathcal{L}'\|$.

Assumption 1. *Each loss function $f_i(x)$ and $f(x,\xi_i)$ is L-smooth such that, for any $x,y\in\mathbb{R}^d$*

$$\|\nabla f_i(x)-\nabla f_i(y)\|\leq L\|x-y\|,$$
$$\|\nabla f(x;\xi_i)-\nabla f(y;\xi_i)\|\leq L\|x-y\|.$$

Assumption 2. *The functions $\psi_j(y_j)$ for all $j\in[m]$ are convex and possibly nonsmooth.*

Assumption 3. *$f(x)$ and $\psi_j(y_j)$ for all $j\in[m]$ are all lower bounded, and let $f^*=\inf_x f(x)>-\infty$ and $\psi_j^*=\inf_{y_j}\psi_j(y_j)>-\infty$.*

Assumption 4. *Matrix A is full row or column rank.*

Assumption 5. *For the online setting, the variance of unbiased stochastic gradient is bounded,* i.e., *$\mathbb{E}[\nabla f(x;\xi)]=\nabla f(x)$ and $\mathbb{E}\|\nabla f(x;\xi)-\nabla f(x)\|^2\leq\sigma^2$ for all $x\in\mathbb{R}^d$.*

Assumption 1 imposes the smoothness on individual loss functions, which is commonly used in the convergence analysis of variance-reduced algorithms [16], [17], [18], [21]. For any $x_1,x_2\in\mathbb{R}^d$, we have $\|\nabla f(x_1)-\nabla f(x_2)\|\leq\|\mathbb{E}[\nabla f(x_1;\xi)-\nabla f(x_2;\xi)]\|\leq\mathbb{E}\|\nabla f(x_1;\xi)-\nabla f(x_2;\xi)\|\leq L\|x_1-x_2\|$. Assumption 1 also implies the full gradient is L-Lipschitz continuous. Assumption 2 is widely used in ADMM methods [51] and proximal methods [9], [25]. For example, the sparse regularization $\psi_1(y_1)=\|y_1\|_1$. Assumptions 3 guarantees the feasibility of the optimization problem, which has been used in the study of nonconvex ADMMs [48], [49]. For example, given a set of training samples $(a_i,b_i)_{i=1}^n$, where $a_i\in\mathbb{R}^d$, $b_i\in\{-1,1\}$, we consider the nonconvex smooth sigmoid loss function $f(x)=\frac{1}{n}\sum_{i=1}^n\frac{1}{1+\exp(b_ia_i^Tx)}$ used in [16], [51]. Clearly, we have $f(x)\geq0$. Let $\psi_1(y_1)=\|y_1\|_1$, we have $\psi_1(y_1)\geq0$ for any $y_1\in\mathbb{R}^p$.

Assumption 4 guarantees the matrix A^TA or AA^T is non-singular, which is commonly used in the convergence analysis of nonconvex ADMM algorithms [48], [49]. Without loss of generality, we will use the full column rank matrix A as in [51]. Assumption 5 shows that the variance of stochastic gradient is bounded in norm, which is widely used in the study of nonconvex methods [9] and zeroth-order methods [11]. Considering the nonconvex cases where there are multiple global minima, we admit that Assumptions 1 and 4 are not mild. Thus, we hope to relax these assumptions in the future work. For example, we do not require full row or column rank of matrix A. Next, we give a useful lemma.

Lemma 1. *Suppose that $\inf_x f(x)=f^*>-\infty$, then*

$$\inf_x\Big\{f(x)-\frac{\beta}{2L}\|\nabla f(x)\|^2\Big\}\geq f^*>-\infty, \tag{16}$$

where $\beta\in[0,1]$.

In the convergence analysis, we admit that the main proofs in our paper can follow the proofs in [51]. However, the main differences between our convergence analysis and the convergence analysis in [51] come from the variances of (zeroth-order) stochastic gradient estimators. For example, in our convergence analysis, the Lyapunov (or potential) functions defined in the proofs are not monotonous, while the Lyapunov (or potential) functions defined in [51] are monotonous. Clearly, our proofs are more complex than the proofs in [51]. For example, we need to choose some appropriate parameters (such as smoothing parameter μ in ZO-SIPDER-ADMM (Coord) algorithm) control the convergence of our algorithms.

In fact, Lemma 1 instead of the bounded norm of objective function, *i.e.*, $\|\nabla f(x)\|^2\leq\delta^2$ or $\|\nabla f(x;\xi)\|^2\leq\delta^2$, which is widely used in the nonconvex (stochastic) ADMM methods [51]. Thus, our convergence analysis uses some milder conditions than the existing convergence analysis in [51].

4.1 Convergence Analysis of ZO-SPIDER-ADMM (CooGE) Algorithm

In the subsection, we study the convergence properties of the ZO-SPIDER-ADMM (CooGE) algorithm.

Theorem 1. *Suppose the sequence* $\{x_k, y_{[m]}^k, \lambda_k)_{k=1}^K$ *be generated from Algorithm 1. Let* $b = q$, $\eta = \frac{\alpha\sigma_{\min}(G)}{4L}$ $(0 < \alpha \leq 1)$, $\rho = \frac{2\sqrt{237}\kappa_G L}{\sigma_{\min}^A \alpha}$, *we have*

$$\mathbb{E}\big[dist(0, \partial L(x_\zeta, y_{[m]}^\zeta, \lambda_\zeta))^2\big] \leq O(\frac{1}{K}) + O(d\mu^2).$$

It implies that the iteration number K *and the smoothing parameter* μ *satisfy* $K = O(\frac{1}{\epsilon})$, $\mu = O(\sqrt{\frac{\epsilon}{d}})$, *then* $(x_{k^*}, y_{[m]}^{k^*}, \lambda_{k^*})$ *is an* ϵ*-approximate stationary point of the problem* (1), *where* $k^* = \arg\min_k \theta_k$.

Remark 1. *Theorem 1 shows that given* $b = q$ *and* $\mu = \frac{1}{\sqrt{dK}}$, *the Algorithm 1 has a convergence rate of* $O(\frac{1}{K})$. *It implies that given* $b = q = \sqrt{n}$, $K = O(\frac{1}{\epsilon})$, $\mu = O(\sqrt{\frac{\epsilon}{d}})$, *the ZO-SPIDER-ADMM (CooGE) algorithm reach a lower function query complexity of* $O(nd + n^{\frac{1}{2}}d\epsilon^{-1})$ *for finding an* ϵ*-stationary point of the* ***finite-sum*** *problem* (1). *Specifically, when* $\mod(k, q) = 0$, *our algorithm needs* $2nd$ *function values to compute a zeroth-order full gradient at each iteration, and needs* $K/q = n^{-\frac{1}{2}}\epsilon^{-1}$ *iterations. When* $\mod(k, q) \neq 0$, *our algorithm needs* $4bd$ *function values to compute the zeroth-order stochastic gradient at each iteration, and needs* $K = \frac{1}{\epsilon}$ *iterations. Meanwhile, our algorithm needs to calculate a zeroth-order full gradient at least. Thus, the ZO-SPIDER-ADMM (CooGE) requires the function query complexity* $nd + 2ndK/q + 4bdK = nd + 2ndn^{-\frac{1}{2}}\epsilon^{-1} + 4n^{-\frac{1}{2}}d\epsilon^{-1} = O(nd + n^{\frac{1}{2}}d\epsilon^{-1})$ *for obtaining an* ϵ*-approximate stationary point.*

4.2 Convergence Analysis of ZO-SPIDER-ADMM (CooGE+UniGE) Algorithm

In the subsection, we study the convergence properties of the ZO-SPIDER-ADMM (CooGE+UniGE) algorithm.

Theorem 2. *Suppose the sequence* $\{x_k, y_{[m]}^k, \lambda_k)_{k=1}^K$ *be generated from the Algorithm 2. Further, let* $b = qd$, $\eta = \frac{\alpha\sigma_{\min}(G)}{4L}$ $(0 < \alpha \leq 1)$ *and* $\rho = \frac{2\sqrt{237}\kappa_G L}{\sigma_{\min}^A \alpha}$, *we have*

$$\mathbb{E}\big[dist(0, \partial L(x_\zeta, y_{[m]}^\zeta, \lambda_\zeta))^2\big] \leq O(\frac{1}{K}) + O(d^2\nu^2) + O(d\mu^2),$$

where $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ *is chosen uniformly randomly from* $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$. *It implies that the iteration number* K, *the smoothing parameter* ν *and the mini-batch size* b_1 *satisfy* $K = O(\frac{1}{\epsilon})$, $\nu = O(\frac{\sqrt{\epsilon}}{d})$, $\mu = O(\frac{\sqrt{\epsilon}}{\sqrt{d}})$ *then* $(x_{k^*}, y_{[m]}^{k^*}, \lambda_{k^*})$ *is an* ϵ*-stationary point of the problem* (1), *where* $k^* = \arg\min_k \theta_k$.

Remark 2. *Theorem 2 shows that given* $b = dq$, $\mu = \frac{1}{\sqrt{dK}}$ *and* $\nu = \frac{1}{d\sqrt{K}}$, *the Algorithm 2 has a convergence rate of* $O(\frac{1}{K})$. *It implies that given* $q = \sqrt{n}$, $K = O(\frac{1}{\epsilon})$, $\mu = O(\sqrt{\frac{\epsilon}{d}})$, $\nu = O(\frac{\sqrt{\epsilon}}{d})$, *the ZO-SPIDER-ADMM (CooGE+UniGE) algorithm reach a lower function query complexity of* $O(nd + n^{\frac{1}{2}}d\epsilon^{-1})$ *for finding an* ϵ*-stationary point of the* ***finite-sum*** *problem* (1). *Specifically, when* $\mod(k, q) = 0$, *our algorithm needs* $2nd$ *function values to compute a zeroth-order full gradient at each iteration, and needs* $K/q = n^{-\frac{1}{2}}\epsilon^{-1}$ *iterations. When* $\mod(k, q) \neq 0$, *our algorithm needs* $4b$ *function values to compute the zeroth-order stochastic gradient at each iteration, and needs* $K = \frac{1}{\epsilon}$ *iterations. Meanwhile, our algorithm needs to calculate a zeroth-order full gradient at least. Thus, the ZO-SPIDER-ADMM (CooGE+UniGE) requires the function query complexity* $nd + 2ndK/q + 4bK = nd + 2ndn^{-\frac{1}{2}}\epsilon^{-1} + 4n^{-\frac{1}{2}}d\epsilon^{-1} = O(nd + n^{\frac{1}{2}}d\epsilon^{-1})$ *for obtaining an* ϵ*-stationary point.*

4.3 Convergence Analysis of ZOO-ADMM+(CooGE) Algorithm

In this subsection, we study the convergence properties of the ZOO-ADMM+(CooGE) algorithm.

Theorem 3. *Suppose the sequence* $\{x_k, y_{[m]}^k, \lambda_k)_{k=1}^K$ *be generated from the Algorithm 3. Further, let* $b_2 = q$, $\eta = \frac{\alpha\sigma_{\min}(G)}{4L}$ $(0 < \alpha \leq 1)$ *and* $\rho = \frac{2\sqrt{237}\kappa_G L}{\sigma_{\min}^A \alpha}$, *we have*

$$\mathbb{E}\big[dist(0, \partial L(x_\zeta, y_{[m]}^\zeta, \lambda_\zeta))^2\big] \leq O(\frac{1}{K}) + O(d\mu^2) + O(\frac{1}{b_1}).$$

It implies that the parameters K, b_1 *and* μ *satisfy* $K = O(\frac{1}{\epsilon})$, $b_1 = O(\frac{1}{\epsilon})$ *and* $\mu = \sqrt{\frac{\epsilon}{d}}$, *then* $(x_{k^*}, y_{[m]}^{k^*}, \lambda_{k^*})$ *is an* ϵ*-approximate stationary point of the problem* (1), *where* $k^* = \arg\min_k \theta_k$.

Remark 3. *Theorem 3 shows that given* $b_2 = q$, $b_1 = K$, $\mu = \frac{1}{\sqrt{dK}}$, *the ZOO-ADMM+(CooGE) algorithm has a convergence rate of* $O(\frac{1}{K})$. *Let* $K = \epsilon^{-1}$, $b_2 = q = \epsilon^{-1/2}$, $b_1 = \epsilon^{-1}$ *and* $\mu = \sqrt{\frac{\epsilon}{d}}$, *the ZOO-ADMM+(CooGE) algorithm reaches a function query complexity of* $2db_1K/q + 4db_2K = O(d\epsilon^{-\frac{3}{2}})$ *for finding an* ϵ*-stationary point of the* ***online*** *problem* (1).

4.4 Convergence Analysis of ZOO-ADMM+ (CooGE+UniGE) Algorithm

In this subsection, we study the convergence properties of the ZOO-ADMM+(CooGE+UniGE) algorithm.

Theorem 4. *Suppose the sequence* $\{x_k, y_{[m]}^k, \lambda_k)_{k=1}^K$ *be generated from Algorithm 4. Further, let* $b_2 = qd$, $\eta = \frac{\alpha\sigma_{\min}(G)}{4L}$ $(0 < \alpha \leq 1)$ *and* $\rho = \frac{2\sqrt{237}\kappa_G L}{\sigma_{\min}^A \alpha}$, *we have*

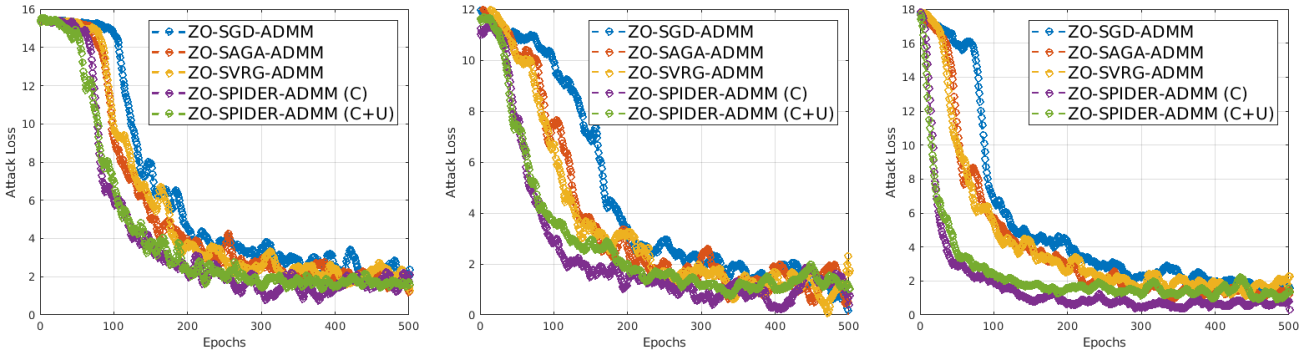
$$\mathbb{E}\big[dist(0, \partial L(x_\zeta, y_{[m]}^\zeta, \lambda_\zeta))^2\big] \leq O(\frac{1}{K}) + O(d^2\nu^2) + O(d\mu^2) + O(\frac{1}{b_1}), \quad (17)$$

where $\{x_\zeta, y_{[m]}^\zeta, \lambda_\zeta\}$ *is chosen uniformly randomly from* $\{x_k, y_{[m]}^k, \lambda_k\}_{k=0}^{K-1}$. *It implies that given* $K = O(\frac{1}{\epsilon})$, $\nu = O(\frac{\sqrt{\epsilon}}{d})$, $\mu = O(\sqrt{\frac{\epsilon}{d}})$, $b_1 = O(\frac{1}{\epsilon})$ *then* $(x_{k^*}, y_{[m]}^{k^*}, \lambda_{k^*})$ *is an* ϵ*-stationary point of the problem* (1), *where* $k^* = \arg\min_k \theta_k$.

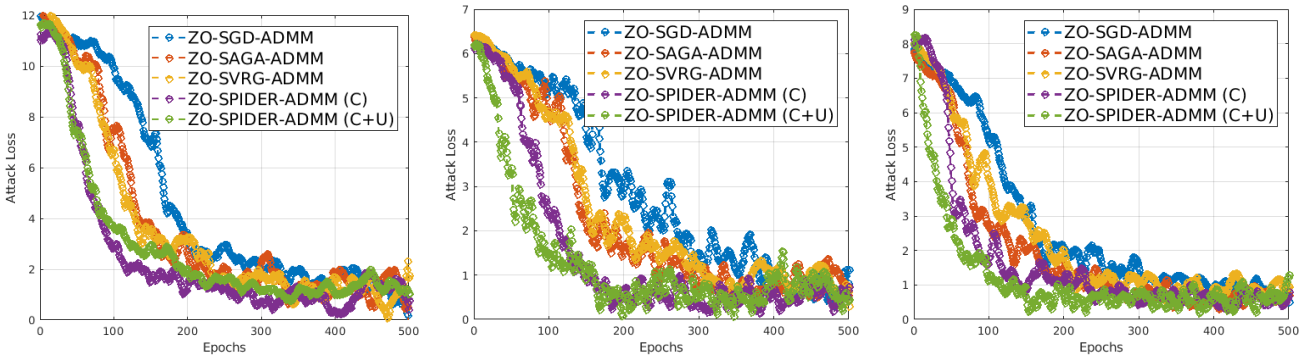
Remark 4. *Theorem 4 shows that given* $b_2 = qd$, $b_1 = K$ *and* $\nu = \frac{1}{d\sqrt{K}}$, *the Algorithm 4 has* $O(\frac{1}{K})$ *convergence rate. Let* $K = \epsilon^{-1}$, $q = \epsilon^{-1/2}$, $b_2 = dq = d\epsilon^{-1/2}$, $b_1 = \epsilon^{-1}$, $\nu = \frac{\sqrt{\epsilon}}{d}$ *and* $\mu = O(\sqrt{\frac{\epsilon}{d}})$, *the ZOO-ADMM+(CooGE+UniGE) algorithm reaches the function query complexity of* $2b_1K/q + 4b_2K = O(d\epsilon^{-\frac{3}{2}})$ *for finding an* ϵ*-stationary point of the* ***online*** *problem* (1).

5 Experiments

In this section, we apply the universal structured adversarial attack on black-box DNNs to demonstrate efficiency of our algorithms. In the finite-sum setting, we compare the proposed ZO-SPIDER-ADMM methods with the existing

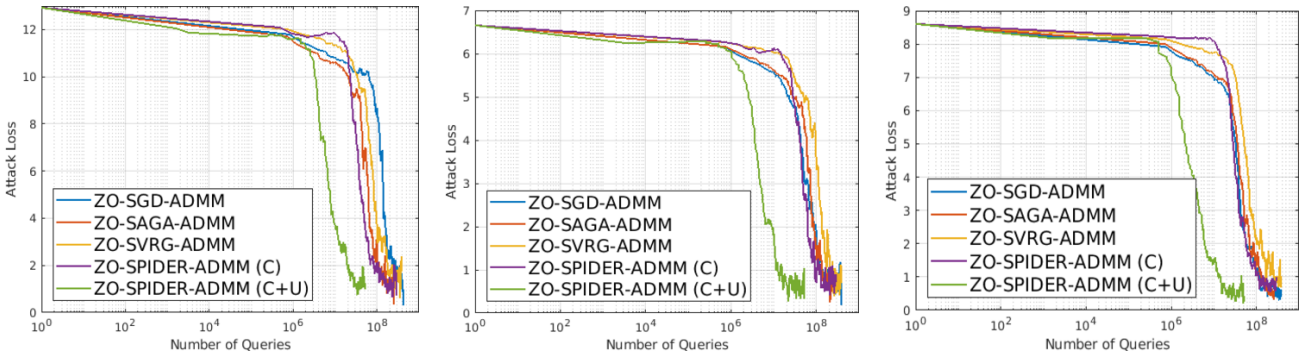


(a) *MNIST* (b) *Fashion-MNIST* (c) *SVHN*



(d) *CIFAR-10* (e) *STL-10* (f) *CINIC-10*

Fig. 2. Attack loss vs. epochs on adversarial attacks black-box DNNs in the finite-sum setting. ZO-SPIDER-ADMM (C) and ZO-SPIDER-ADMM (C+U) represent ZO-SPIDER-ADMM (CooGE) and ZO-SPIDER-ADMM (CooGE+UniGE), respectively.



(a) *CIFAR-10* (b) *STL-10* (c) *CINIC-10*

Fig. 3. Attack loss vs. number of queries on adversarial attacks black-box DNNs in the finite-sum setting.

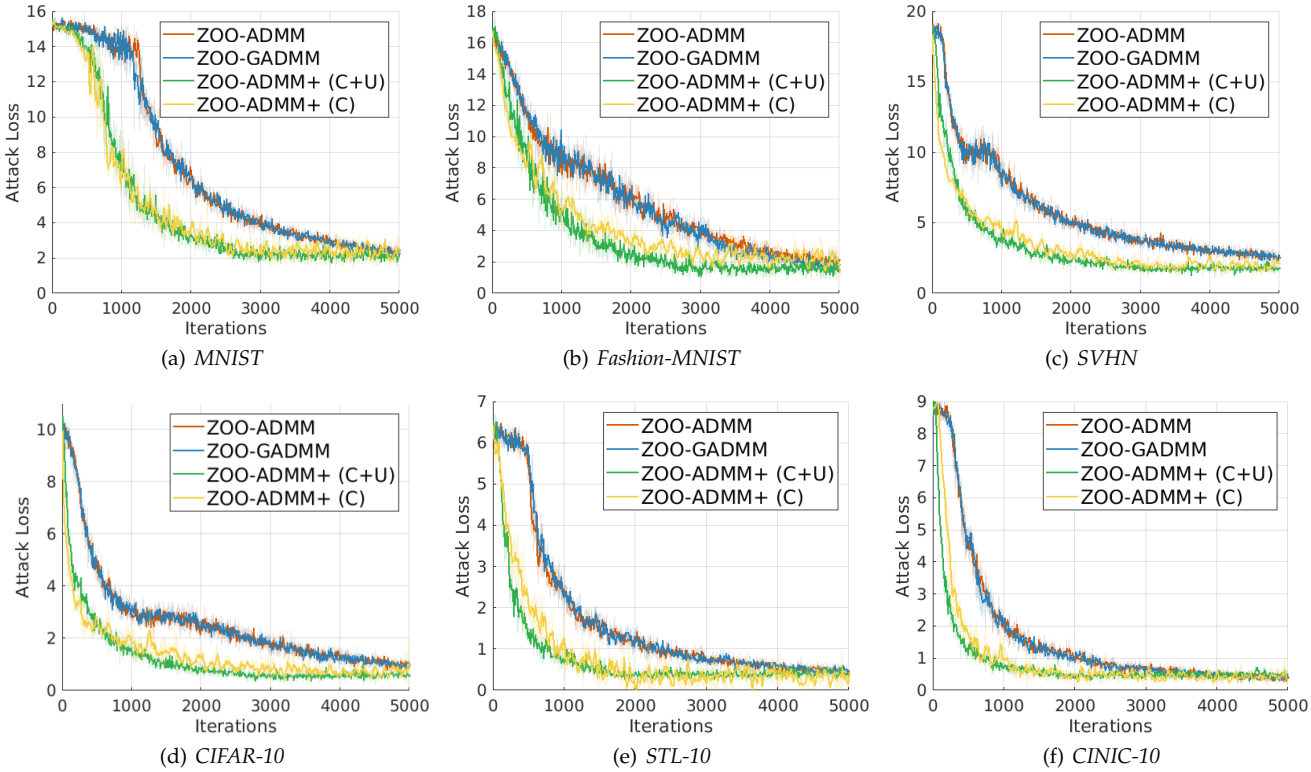
TABLE 2
Four Benchmark Datasets for Attacking Black-Box DNNs

datasets	#test samples	#dimension	#classes
MNIST	10,000	28×28	10
Fashion-MNIST	10,000	28×28	10
SVHN	26,032	$32\times32\times3$	10
CIFAR-10	10,000	$32\times32\times3$	10
STL-10 (resized)	8,000	$32\times32\times3$	10
CINIC-10	90,000	$32\times32\times3$	10

zeroth-order stochastic ADMM methods (i.e., ZO-SAGA-ADMM and ZO-SVRG-ADMM [13]), and the zeroth-order stochastic ADMM without variance reduction (ZO-SGD-ADMM). In the online setting, we compare the proposed ZOO-ADMM+ methods with the ZOO-ADMM [10] and ZO-GADM [11] methods.

5.1 Experimental Setups

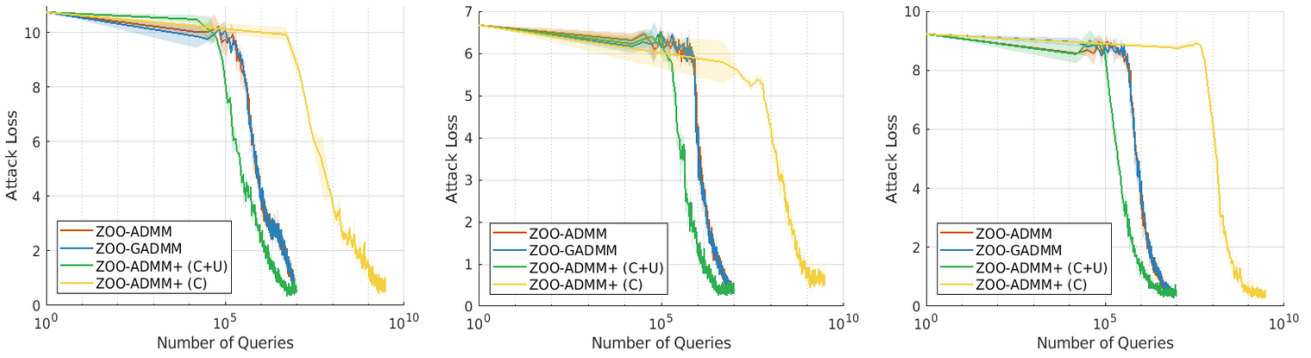
In this subsection, we apply our algorithms to generate adversarial examples to attack the pre-trained black-box DNNs, whose parameters are hidden from us and only its outputs are accessible. In the experiment, we mainly focus on finding a universal structured perturbation to fool the DNNs from multiple images as in [13], [54]. Given the samples $\{a_i \in \mathbb{R}^d,\ l_i \in \{1,2,\cdots,c\}\ i = 1,2,\ldots,n\}$, as



(a) *MNIST* (b) *Fashion-MNIST* (c) *SVHN*

(d) *CIFAR-10* (e) *STL-10* (f) *CINIC-10*

Fig. 4. Attack loss vs. epochs on adversarial attacks black-box DNNs in the online setting. ZOO-ADMM+ (C) and ZOO-ADMM+ (C+U) represent ZOO-ADMM+ (CooGE) and ZOO-ADMM+ (CooGE+UniGE), respectively.



(a) *CIFAR-10* (b) *STL-10* (c) *CINIC-10*

Fig. 5. Attack loss vs. number of queries on adversarial attacks black-box DNNs in the online setting.

in [13], we solve the following problem

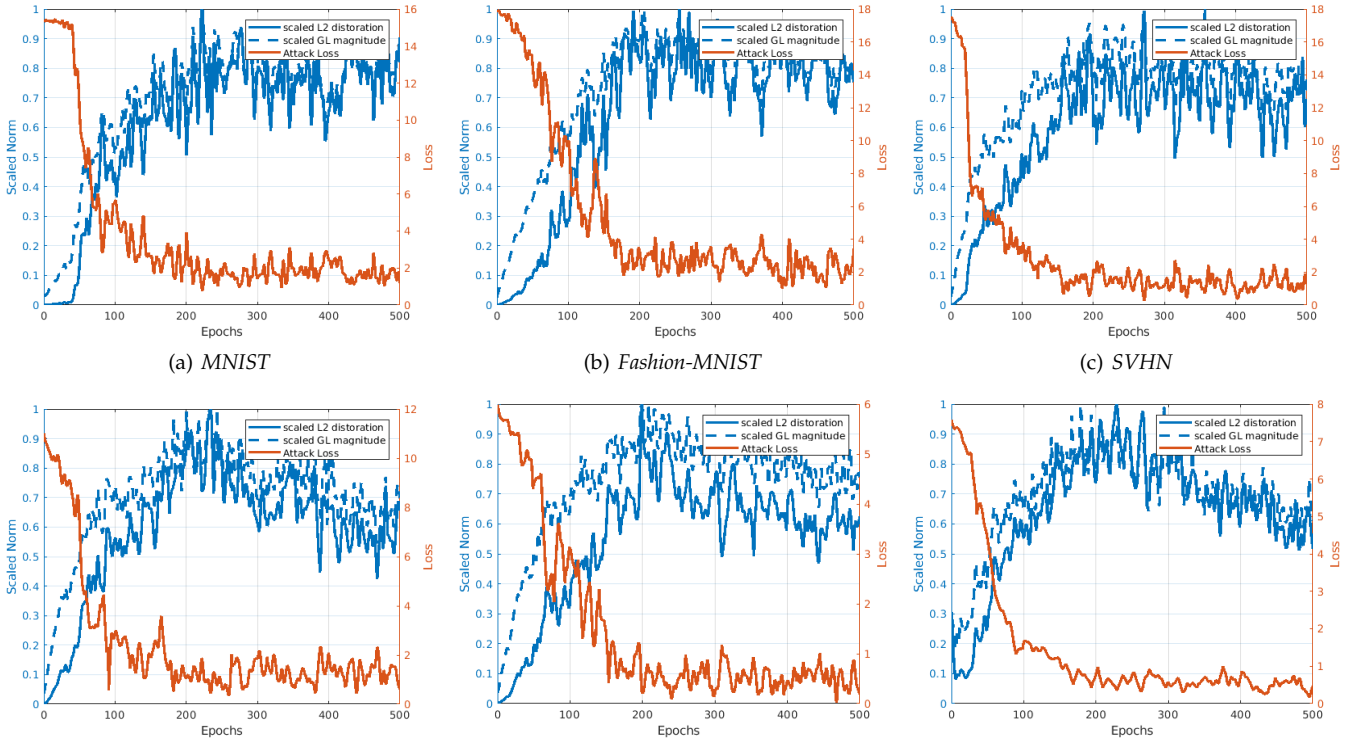
$$\min_{x\in\mathbb{R}^d}\Big\{\frac{1}{n}\sum_{i=1}^{n}\max\big\{F_{l_i}(a_i+x)-\max_{j\neq l_i}F_j(a_i+x),0\big\} + \tau_1\sum_{p=1}^{P}\sum_{q=1}^{Q}\|x_{\mathcal{G}_{p,q}}\|+\tau_2\|x\|^2+\tau_3 h(x)\Big\}, \quad (18)$$

where $x\in\mathbb{R}^d$ denotes a universal structured perturbation, and $\{a_i,l_i\}_{i=1}^n$ are the input images and the corresponding labels. Here $F(a)$ represents the final layer output before softmax of neural network, and τ_i $(i=1,2,3)$ are non-negative tuning parameters. In the above problem (18), the first penalty $\sum_{p,q}\|x_{G_{p,q}}\|$ is to penalize structured permutations as in [26], where the overlapping groups $\{\{\mathcal{G}_{p,q}\}_{p=1}^P\}_{q=1}^Q$ generate from dividing an image into sub-groups of pixels. The second penalty $\|x\|^2$ is to ensure the permutation is small, so that it become harder to detect. The third penalty $h(x)$ is to ensure the attacked image $a_i+x\in[0,1]^d$ is a valid image, defined as

$$h(x)=\begin{cases}0, & \text{if } \{a_i+x\in[0,1]^d\}_{i=1}^n \text{ and } \|x\|_\infty\le\varepsilon\\ \infty, & \text{otherwise}\end{cases}$$

Let $f_i(x)=\max\big\{F_{l_i}(a_i+x)-\max_{j\neq l_i}F_j(a_i+x),0\big\}$, then we rewrite the problem (18) in the following form:

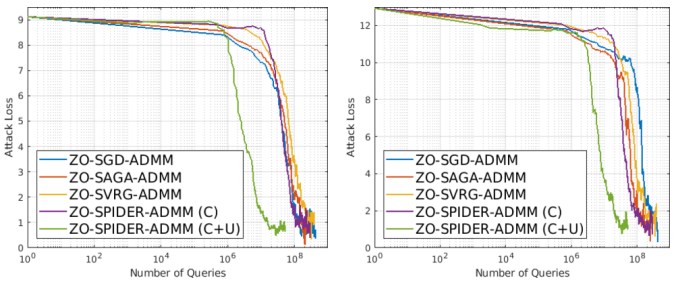
$$\min\ \frac{1}{n}\sum_{i=1}^{n}f_i(x)+\tau_1\sum_{j=1}^{PQ}\|y_{j,\mathcal{G}_j}\|+\tau_2\|z\|^2+\tau_3 h(w), \quad (19)$$
$$\text{s.t. } z=x,\ w=x,\ y_j=x,\ \text{for } j=1,\cdots,PQ$$



(a) *MNIST* (b) *Fashion-MNIST* (c) *SVHN*

(d) *CIFAR-10* (e) *STL-10* (f) *CINIC-10*

Fig. 6. Relationship between attack loss and norms of perturbation (L_2 norm distoration and Group lasso magnitude) on adversarial attacks black-box DNNs.



(a) *ResNet-20* (b) *5-layer DNN*

Fig. 7. The attack loss for different models on the CIFAR-10 dataset.

where $y_{j,\mathcal{G}_j}$ denotes the sub-vector of $y_j \in \mathbb{R}^d$ with indices given by group $\mathcal{G}_j$. Clearly, the above problem (19) can easy be rewritten as the form of the problem (1).

In the experiment, we use the pre-trained DNN models on four benchmark datasets in Table 2 as the target black-box models. Specifically, the pre-trained DNNs on MNIST, Fashion-MNIST, SVHN, CIFAR-10, STL-10 [55] and CINIC-10 [56] can attain 99.4%, 91.8%, 93.2%, 80.8%, 71.8% and 83.3% test accuracy, respectively. For MNIST and Fashion-MNIST, a 5-layer DNN (3 conv layers and 2 dense layers) is used. For the CINIC-10 dataset, ResNet-32 is used. For the rest dataset, an 7-layer (5 conv layers and 2 dense layers) DNN is used.

In the experiment, we set the parameters $\varepsilon = 0.4$, $\tau_1 = 1$, $\tau_2 = 2$ and $\tau_3 = 1$. In the zeroth-order gradient estimator, we choose the smoothing parameters $\mu = \frac{1}{\sqrt{dk}}$ and $\nu = \frac{1}{d\sqrt{k}}$. For all datasets, the kernel size for overlapping group lasso is set to 3×3 and the stride is one. In the **finite-sum** setting, we select 40 samples from each class, and choose 4 as the batch size. In the **online** setting, we select 400 samples from each class, and choose 10 as the batch size.

5.2 Experimental Results

Figure 2 illustrates that, in the finite-sum setting, the attack losses (*i.e.* the first term of the problem (18)) of our ZO-SPIDER-ADMM algorithms faster decrease than the other algorithms, as the epoch (*i.e.*, 10 iterations) increases. Figure 4 shows that, in the online setting, the attack losses of our ZOO-ADMM+ algorithms also faster decrease than the ZOO-ADMM and ZO-GADM algorithms, as the number of iteration increases. These results demonstrate that our algorithms have faster convergence than the existing zeroth-order algorithms in solving the above complex problem (18).

Figs. 3 and 5 show the results on attack losses vs number of queries in the finite-sum and stochastic cases, respectively. In the finite-sum setting, our ZO-SPIDER-ADMM (C+U) has the lowest query costs. In the online setting, our ZOO-ADMM+ (C+U) has the lowest query costs. ZO-SPIDER-ADMM (C) has similar query costs compared to other methods in the finite sum setting, and ZOO-ADMM+ (C) has the largest query costs in the online learning setting. It is because that all comparison methods use cheaper zeroth order gradient estimators, while our ZO-SPIDER-ADMM (C) and ZOO-ADMM+ (C) methods use the expensive CooGE gradient estimator. Fig. 7 shows the attack losses vs number of queries in different models (i.e., ResNet-20 and 5-layer DNN) on the CIFAR-10 dataset. From Fig. 7, we can

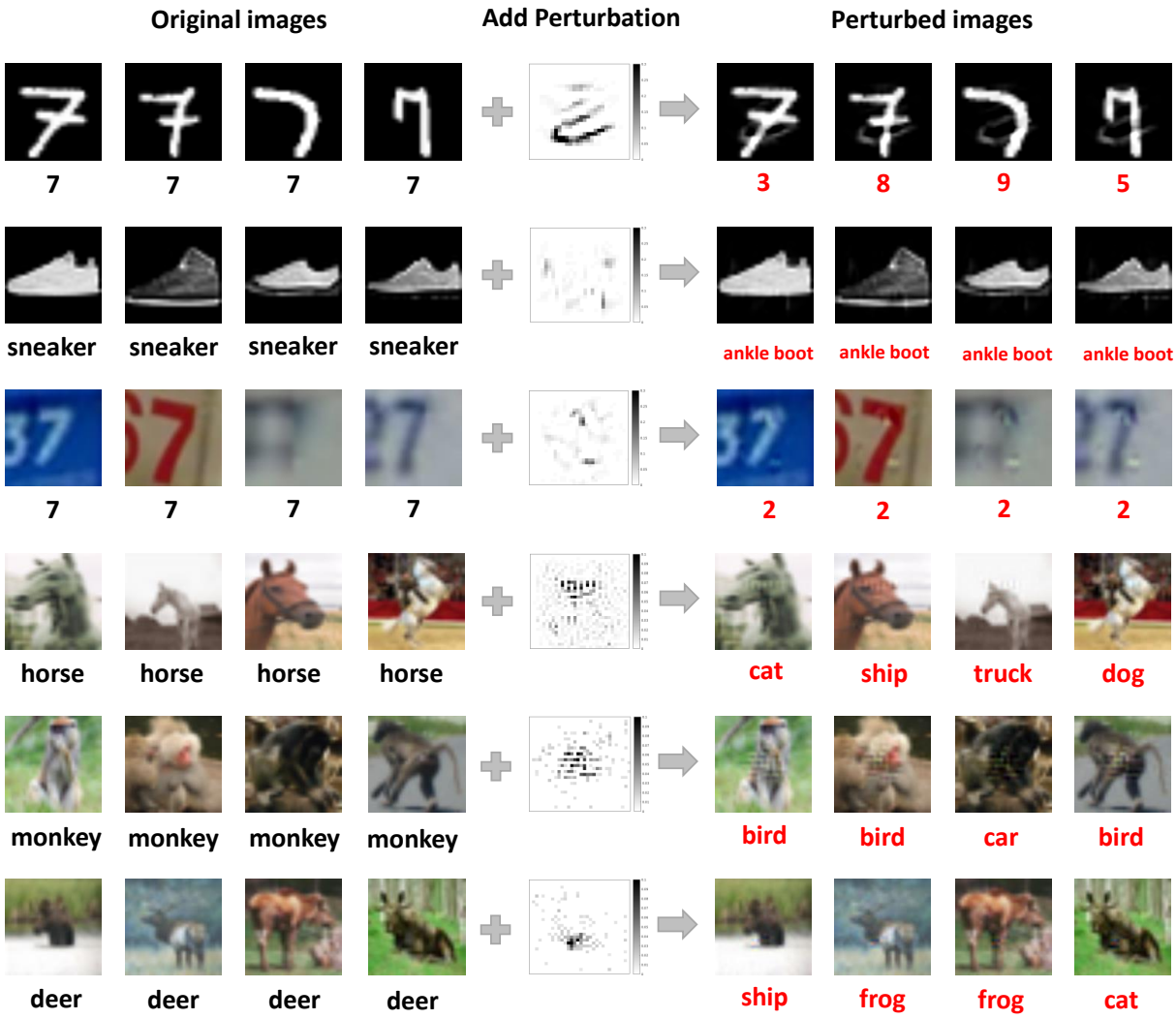


Fig. 8. Group-sparsity perturbations are learned from *MNIST*, *Fashion-MNIST*, *SVHN*, *CIFAR-10*, *STL-10* and *CINIC-10* (from top to bottom) datasets. Black and red labels denote the initial label and the label after attack, respectively.

find that different models only change the numeric results, but the overall tendency is similar.

Fig. 6 shows that relationship between the attack loss and the norms of perturbation (L_2 norm distoration and Group lasso magnitude), which conforms to the phenomenon of adversarial attack. From Fig. 8, we can find that our algorithms can learn some interpretable structured perturbations, which can successfully attack the corresponding DNNs.

6 Conclusions

In this paper, we proposed a class of faster zeroth-order stochastic ADMM (ZO-SPIDER-ADMM) methods to solve the problem (1). We proved that the ZO-SPIDER-ADMM achieves a lower function query complexity of $O(dn + dn^{\frac{1}{2}}\epsilon^{-1})$ for finding an ϵ-stationary point, which improves the existing zeroth-order ADMM methods by a factor $O(d^{\frac{1}{3}}n^{\frac{1}{6}})$. At the same time, we extend the ZO-SPIDER-ADMM method to the online setting, and propose a class of faster zeroth-order online ADMM (ZOO-ADMM+) methods. Moreover, we proved that the ZOO-ADMM+ reaches a lower function query complexity of $O(d\epsilon^{-\frac{3}{2}})$, which improves the existing best result by a factor of $O(\epsilon^{-\frac{1}{2}})$.

Acknowledgments

We would like to express our gratitude to both the editor and the anonymous reviewers for their valuable comments that significantly improved the quality of our paper. F. Huang was a postdoctoral researcher at the University of Pittsburgh. S.Q. Gao and H. Huang were partially supported by U.S. NSF IIS 1838627, 1837956, 1956002, 2211492, CNS 2213701, CCF 2217003, DBI 2225775.

References

[1] X. Hu, L. Prashanth, A. György, and C. Szepesvari, "(bandit) convex optimization with biased noisy gradient oracles," in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 819–828.

[2] J. Larson, M. Menickelly, and S. M. Wild, "Derivative-free optimization methods," *Acta Numerica*, vol. 28, pp. 287–404, 2019.

[3] A. Agarwal, O. Dekel, and L. Xiao, "Optimal algorithms for online convex optimization with multi-point bandit feedback." in *COLT*. Citeseer, 2010, pp. 28–40.

[4] K. Choromanski, M. Rowland, V. Sindhwani, R. E. Turner, and A. Weller, "Structured evolution with compact architectures for scalable policy optimization," *arXiv preprint arXiv:1804.02395*, 2018.

[5] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," in *Workshop on Artificial Intelligence and Security*. ACM, 2017, pp. 15–26.

[6] S. Liu, B. Kailkhura, P.-Y. Chen, P. Ting, S. Chang, and L. Amini, "Zeroth-order stochastic variance reduction for nonconvex optimization," in *NIPS*, 2018, pp. 3731–3741.

[7] Y. Nesterov and V. G. Spokoiny, "Random gradient-free minimization of convex functions," *Foundations of Computational Mathematics*, vol. 17, pp. 527–566, 2017.

[8] S. Ghadimi and G. Lan, "Stochastic first-and zeroth-order methods for nonconvex stochastic programming," *SIAM Journal on Optimization*, vol. 23, no. 4, pp. 2341–2368, 2013.

[9] S. Ghadimi, G. Lan, and H. Zhang, "Mini-batch stochastic approximation methods for nonconvex stochastic composite optimization," *Mathematical Programming*, vol. 155, no. 1-2, pp. 267–305, 2016.

[10] S. Liu, J. Chen, P.-Y. Chen, and A. Hero, "Zeroth-order online alternating direction method of multipliers: Convergence analysis and applications," in *AISTATS*, vol. 84, 2018, pp. 288–297.

[11] X. Gao, B. Jiang, and S. Zhang, "On the information-adaptive variants of the admm: an iteration complexity perspective," *Journal of Scientific Computing*, vol. 76, no. 1, pp. 327–363, 2018.

[12] K. Balasubramanian and S. Ghadimi, "Zeroth-order (non)-convex stochastic optimization via conditional gradient and gradient updates," in *Advances in Neural Information Processing Systems*, 2018, pp. 3455–3464.

[13] F. Huang, S. Gao, S. Chen, and H. Huang, "Zeroth-order stochastic alternating direction method of multipliers for nonconvex nonsmooth optimization," in *IJCAI*, 2019, pp. 2549–2555.

[14] L. Liu, M. Cheng, C.-J. Hsieh, and D. Tao, "Stochastic zeroth-order optimization via variance reduction method," *CoRR*, vol. abs/1805.11811, 2018.

[15] R. Johnson and T. Zhang, "Accelerating stochastic gradient descent using predictive variance reduction," in *NIPS*, 2013, pp. 315–323.

[16] Z. Allen-Zhu and E. Hazan, "Variance reduction for faster nonconvex optimization," in *International conference on machine learning*. PMLR, 2016, pp. 699–707.

[17] S. J. Reddi, A. Hefny, S. Sra, B. Poczos, and A. Smola, "Stochastic variance reduction for nonconvex optimization," in *International conference on machine learning*. PMLR, 2016, pp. 314–323.

[18] C. Fang, C. J. Li, Z. Lin, and T. Zhang, "Spider: Near-optimal nonconvex optimization via stochastic path-integrated differential estimator," in *Advances in Neural Information Processing Systems*, 2018, pp. 689–699.

[19] K. Ji, Z. Wang, Y. Zhou, and Y. Liang, "Improved zeroth-order variance reduced algorithms and analysis for nonconvex optimization," in *International Conference on Machine Learning*, 2019, pp. 3100–3109.

[20] L. M. Nguyen, J. Liu, K. Scheinberg, and M. Takáč, "Sarah: A novel method for machine learning problems using stochastic recursive gradient," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org, 2017, pp. 2613–2621.

[21] Z. Wang, K. Ji, Y. Zhou, Y. Liang, and V. Tarokh, "Spiderboost and momentum: Faster variance reduction algorithms," in *Advances in Neural Information Processing Systems*, 2019, pp. 2403–2413.

[22] X. Lian, H. Zhang, C.-J. Hsieh, Y. Huang, and J. Liu, "A comprehensive linear speedup analysis for asynchronous stochastic parallel optimization from zeroth-order to first-order," in *Advances in Neural Information Processing Systems*, 2016, pp. 3054–3062.

[23] B. Gu, Z. Huo, C. Deng, and H. Huang, "Faster derivative-free stochastic algorithm for shared memory machines," in *ICML*, 2018, pp. 1807–1816.

[24] D. Hajinezhad, M. Hong, and A. Garcia, "Zeroth order nonconvex multi-agent optimization over networks," *arXiv preprint arXiv:1710.09997*, 2017.

[25] F. Huang, B. Gu, Z. Huo, S. Chen, and H. Huang, "Faster gradient-free proximal stochastic methods for nonconvex nonsmooth optimization," in *AAAI*, 2019, pp. 1503–1510.

[26] K. Xu, S. Liu, P. Zhao, P.-Y. Chen, H. Zhang, D. Erdogmus, Y. Wang, and X. Lin, "Structured adversarial attack: Towards general implementation and better interpretability," *arXiv preprint arXiv:1808.01664*, 2018.

[27] J. Wright, A. Ganesh, S. R. Rao, Y. Peng, and Y. Ma, "Robust principal component analysis: Exact recovery of corrupted low-rank matrices via convex optimization," in *NIPS*, 2009.

[28] E. J. Candès, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis?" *Journal of the ACM (JACM)*, vol. 58, no. 3, pp. 1–37, 2011.

[29] N. Simon, J. Friedman, T. Hastie, and R. Tibshirani, "A sparse-group lasso," *Journal of computational and graphical statistics*, vol. 22, no. 2, pp. 231–245, 2013.

[30] S. Kim, K.-A. Sohn, and E. P. Xing, "A multivariate regression approach to association analysis of a quantitative trait network," *Bioinformatics*, vol. 25, no. 12, pp. i204–i212, 2009.

[31] H. Ouyang, N. He, L. Tran, and A. G. Gray, "Stochastic alternating direction method of multipliers." *ICML*, vol. 28, pp. 80–88, 2013.

[32] D. Gabay and B. Mercier, "A dual algorithm for the solution of nonlinear variational problems via finite element approximation," *Computers & Mathematics with Applications*, vol. 2, no. 1, pp. 17–40, 1976.

[33] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

[34] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM journal on imaging sciences*, vol. 2, no. 1, pp. 183–202, 2009.

[35] B. He and X. Yuan, "On the o(1/n) convergence rate of the douglas–rachford alternating direction method," *SIAM Journal on Numerical Analysis*, vol. 50, no. 2, pp. 700–709, 2012.

[36] B. He, F. Ma, and X. Yuan, "Convergence study on the symmetric version of admm with larger step sizes," *SIAM Journal on Imaging Sciences*, vol. 9, no. 3, pp. 1467–1501, 2016.

[37] R. Nishihara, L. Lessard, B. Recht, A. Packard, and M. Jordan, "A general analysis of the convergence of admm," in *International Conference on Machine Learning*, 2015, pp. 343–352.

[38] Y. Xu, M. Liu, Q. Lin, and T. Yang, "Admm without a fixed penalty parameter: Faster convergence with new adaptive penalization," in *Advances in Neural Information Processing Systems*, 2017, pp. 1267–1277.

[39] C. Lu, J. Feng, S. Yan, and Z. Lin, "A unified alternating direction method of multipliers by majorization minimization," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 3, pp. 527–541, 2017.

[40] T. Suzuki, "Dual averaging and proximal gradient descent for online alternating direction multiplier method." in *ICML*, 2013, pp. 392–400.

[41] ——, "Stochastic dual coordinate ascent with alternating direction method of multipliers," in *ICML*, 2014, pp. 736–744.

[42] S. Zheng and J. T. Kwok, "Fast and light stochastic admm," in *IJCAI*, 2016.

[43] Y. Liu, F. Shang, and J. Cheng, "Accelerated variance reduced stochastic admm," in *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[44] Y. Liu, F. Shang, H. Liu, L. Kong, J. Licheng, and Z. Lin, "Accelerated variance reduction stochastic admm for large-scale machine learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[45] G. Taylor, R. Burmeister, Z. Xu, B. Singh, A. Patel, and T. Goldstein, "Training neural networks without gradients: a scalable admm approach," in *ICML*, 2016, pp. 2722–2731.

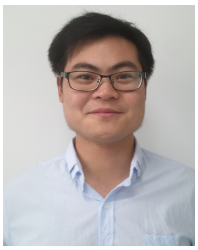
[46] F. Wang, W. Cao, and Z. Xu, "Convergence of multi-block bregman admm for nonconvex composite problems," *arXiv preprint arXiv:1505.03063*, 2015.

[47] Y. Wang, W. Yin, and J. Zeng, "Global convergence of admm in nonconvex nonsmooth optimization," *Journal of Scientific Computing*, pp. 1–35, 2015.

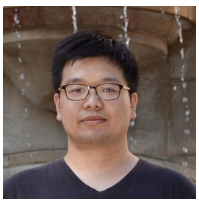
[48] M. Hong, Z.-Q. Luo, and M. Razaviyayn, "Convergence analysis of alternating direction method of multipliers for a family of nonconvex problems," *SIAM Journal on Optimization*, vol. 26, no. 1, pp. 337–364, 2016.

[49] B. Jiang, T. Lin, S. Ma, and S. Zhang, "Structured nonconvex and nonsmooth optimization: algorithms and iteration complexity analysis," *Computational Optimization and Applications*, vol. 72, no. 1, pp. 115–157, 2019.

[50] F. Huang, S. Chen, and Z. Lu, "Stochastic alternating direction method of multipliers with variance reduction for nonconvex optimization," *arXiv preprint arXiv:1610.02758*, 2016.
[51] F. Huang, S. Chen, and H. Huang, "Faster stochastic alternating direction method of multipliers for nonconvex optimization," in *International Conference on Machine Learning*, 2019, pp. 2839–2848.
[52] J. C. Duchi, M. I. Jordan, M. J. Wainwright, and A. Wibisono, "Optimal rates for zero-order convex optimization: The power of two function evaluations," *IEEE TIT*, vol. 61, no. 5, pp. 2788–2806, 2015.
[53] C. Chen, B. He, Y. Ye, and X. Yuan, "The direct extension of admm for multi-block convex minimization problems is not necessarily convergent," *Mathematical Programming*, vol. 155, no. 1-2, pp. 57–79, 2016.
[54] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1765–1773.
[55] A. Coates, A. Ng, and H. Lee, "An analysis of single-layer networks in unsupervised feature learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011, pp. 215–223.
[56] L. N. Darlow, E. J. Crowley, A. Antoniou, and A. J. Storkey, "Cinic-10 is not imagenet or cifar-10," *arXiv preprint arXiv:1810.03505*, 2018.



Feihu Huang received the PhD degree in computer science and technology from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in Dec. 2017. He was a Postdoctoral Researcher at University of Pittsburgh, USA, from Sep. 2018 to Jul. 2022. He is currently a full Professor at Nanjing University of Aeronautics and Astronautics, Nanjing, China. His current research interests include machine learning and optimization.



Shangqian Gao received the B.Eng. degree in Electrical Engineering from Xidian University, China, in 2015 and M.S. degree in Computer System Engineering from Northeastern University, USA, in 2017. He is pursuing the PhD degree at the University of Pittsburgh under the supervision of Dr. Heng Huang. His research interests include computer vision, machine learning and deep learning.

Jian Pei (F'14) is an Arthur S. Pearse Distinguished Professor of Computer Science, Duke University, USA. He is renowned for his productive research in the general areas of data science, big data, data mining, and database systems. He has published prolifically and his publications have been extensively cited. He is recognized as a fellow of the Royal Society of Canada, the Canadian Academy of Engineering, the ACM and the IEEE. He received many prestigious awards, such as the ACM SIGKDD Innovation Award, the ACM SIGKDD Service Award, and the IEEE ICDM Research Award. He has extensive experience in industry RD, strategy consulting, and business operation. Particularly, he is an expert in business data strategies, data products, data assets and data capitalization.

Heng Huang received both B.S. and M.S. degrees from Shanghai Jiao Tong University, Shanghai, China, in 1997 and 2001, respectively. He received the Ph.D. degree in Computer Science from Dartmouth College in 2006. He is a Brendan Iribe Endowed Professor in Computer Science at the University of Maryland College Park. His research interests include machine learning, data mining, computer vision, biomedical data science.